topic_2_functor_pattern

Theory, Edge Cases & Code Examples

- ✓ What is a Functor?
- ✓ Why Functors Matter in Autonomous Driving
- ✓ Functor vs Function Pointer vs Lambda
- ✓ Thread-Safety with Shared State
- ✓ Const-Correctness in Operator()

→ Swipe to learn →

topic_2_functor_pattern

Theory

📌 What is a Functor?

A **functor** (function object) is a class or struct that overloads the `operator()`, making instances of the class callable like functions. Unlike regular functions, functors can maintain **internal state** between calls, making them powerful tools for stateful computations, callbacks, and algorithm customization.

```
class Multiplier {
    int factor;
public:
    Multiplier(int f) : factor(f) {}

    int operator()(int x) const {
        return x * factor;
    }
};

// Usage
Multiplier times3(3);
int result = times3(10);  // result = 30
```

topic_2_functor_pattern

Theory

📌 Why Functors Matter in Autonomous Driving

In autonomous vehicle systems, functors are extensively used for:

1. **Sensor Data Processing**: Stateful filters that maintain calibration parameters
2. **Prediction Models**: Callable objects that cache intermediate computations
3. **Control Algorithms**: PID controllers with internal state (error accumulation, previous values)
4. **STL Algorithms**: Custom predicates and comparators for sorting/filtering sensor data
5. **Event Handling**: Callbacks that track event history

Key Advantages:
- **State Preservation**: Unlike function pointers, functors remember state across calls
- **Inlining**: Compiler can inline `operator()`, making them faster than function pointers
- **Type Safety**: Strong typing at compile-time
- **Flexibility**: Can be passed to STL algorithms (`std::sort`, `std::transform`, etc.)

topic_2_functor_pattern

Theory

📌 Functor vs Function Pointer vs Lambda

Feature	Functor	Function Pointer	Lambda
State Preservation	✅ Yes	❌ No	✅ Yes (capture)
Inlining	✅ Yes	❌ No	✅ Yes
Type Safety	✅ Strong	⚠️ Weak	✅ Strong
Reusability	✅ High	✅ High	⚠️ Limited
STL Compatible	✅ Yes	✅ Yes	✅ Yes
Syntax Complexity	⚠️ More boilerplate	✅ Simple	✅ Concise

When to Use Functors:
- When you need **persistent state** across multiple calls
- When **performance** is critical (inline optimization)
- When you want to **encapsulate complex logic** in a reusable class
- When you need **multiple different behaviors** from the same functor (operator overloading)

---

topic_2_functor_pattern

Edge Cases

📌 1. Thread-Safety with Shared State

Problem: Multiple threads accessing the same functor instance can cause data races.

```
// ❌ NOT Thread-Safe
class Counter {
    int count = 0;
public:
    void operator()() {
        count++;  // ❌ Data race!
    }
    int getCount() const { return count; }
};

// ✅ Thread-Safe with Atomic
class SafeCounter {
    std::atomic<int> count{0};
public:
    void operator()() {
        count++;  // ✅ Atomic operation
    }
    int getCount() const { return count.load(); }
};

// ✅ Thread-Safe with Mutex
class MutexCounter {
    int count = 0;
    mutable std::mutex mtx;
public:
    void operator()() {
        std::lock_guard<std::mutex> lock(mtx);
        count++;
    }
    int getCount() const {
        std::lock_guard<std::mutex> lock(mtx);
        return count;
    }
};
```

Key Takeaway: Always protect mutable state in functors with `std::atomic` or `std::mutex` when used in multithreaded contexts.

topic_2_functor_pattern

Edge Cases

📌 2. Const-Correctness in Operator()

Problem: Forgetting `const` on `operator()` can prevent usage with const functors or STL algorithms.

```
// ❌ Non-const operator() - limits usability
class Adder {
    int offset;
public:
    Adder(int o) : offset(o) {}

    int operator()(int x) {  // ❌ Non-const
        return x + offset;
    }
};

// This fails:
const Adder add5(5);
// int result = add5(10);  // ❌ Error: cannot call non-const function

// ✅ Const-correct version
class ConstAdder {
    int offset;
public:
    ConstAdder(int o) : offset(o) {}

    int operator()(int x) const {  // ✅ Const-qualified
        return x + offset;
    }
};

const ConstAdder add5(5);
int result = add5(10);  // ✅ Works!
```

Key Takeaway: Make `operator()` `const` when it doesn't modify internal state. Use `mutable` for cache/stats that need to change even in const contexts.

📌 3. Copy Semantics and STL Algorithm Behavior

Problem: STL algorithms may copy functors, leading to unexpected state loss.

```
// ❌ Surprising behavior with copies
class CallCounter {
    int calls = 0;
public:
    void operator()(int x) {
        calls++;
        std::cout << "Call #" << calls << ": " << x << "\n";
    }
    int getCalls() const { return calls; }
};

CallCounter counter;
std::vector<int> vec = {1, 2, 3, 4};
std::for_each(vec.begin(), vec.end(), counter);  // ❌ counter is copied!

std::cout << "Total calls: " << counter.getCalls();  // ❌ Prints 0!
// The original counter was never modified - std::for_each worked on a copy

// ✅ Fix 1: Use std::ref to avoid copying
std::for_each(vec.begin(), vec.end(), std::ref(counter));
std::cout << "Total calls: " << counter.getCalls();  // ✅ Prints 4

// ✅ Fix 2: Return the functor from std::for_each
auto result = std::for_each(vec.begin(), vec.end(), CallCounter());
std::cout << "Total calls: " << result.getCalls();  // ✅ Prints 4
```

Key Takeaway: STL algorithms copy functors by default. Use `std::ref` to pass by reference, or retrieve the returned functor from algorithms like `std::for_each`.

topic_2_functor_pattern

Edge Cases

📌 4. Memoization Cache Invalidation (1/2)

Problem: Cached results may become stale if dependencies change.

```
// ❌ Cache without invalidation
class ExpensiveComputation {
    mutable std::unordered_map<int, int> cache;
    int externalFactor;  // ❌ What if this changes?

public:
    ExpensiveComputation(int factor) : externalFactor(factor) {}

    int operator()(int x) const {
        auto it = cache.find(x);
        if (it != cache.end()) {
            return it->second;
        }

        int result = x * x * externalFactor;  // Expensive computation
        cache[x] = result;
        return result;
    }
};

// ✅ Cache with invalidation
class SafeCachedComputation {
    mutable std::unordered_map<int, int> cache;
    int externalFactor;

public:
    SafeCachedComputation(int factor) : externalFactor(factor) {}

    void setFactor(int factor) {
        if (externalFactor != factor) {
            cache.clear();  // ✅ Invalidate cache on dependency change
            externalFactor = factor;
        }
    }

    int operator()(int x) const {
        auto it = cache.find(x);
        if (it != cache.end()) {
// ...continues on next slide
```

📌 4. Memoization Cache Invalidation (2/2)

[Code continues across slides]

```
// ...continued from previous slide
            return it->second;
        }

        int result = x * x * externalFactor;
        cache[x] = result;
        return result;
    }
};
```

Key Takeaway: When caching in functors, provide cache invalidation mechanisms when dependencies change.

📌 5. Double Lookup in Hash-Based Memoization

Problem: Using `.count()` followed by `[]` performs two hash lookups.

```
// ❌ Inefficient double lookup
std::unordered_map<int, int> cache;

int operator()(int x) const {
    if (cache.count(x)) {           // ❌ First lookup
        return cache[x];            // ❌ Second lookup
    }
    int result = compute(x);
    cache[x] = result;
    return result;
}

// ✅ Efficient single lookup with find()
int operator()(int x) const {
    auto it = cache.find(x);        // ✅ Single lookup
    if (it != cache.end()) {
        return it->second;          // ✅ Use iterator
    }
    int result = compute(x);
    cache[x] = result;
    return result;
}

// ✅ Alternative: try_emplace (C++17)
int operator()(int x) const {
    auto [it, inserted] = cache.try_emplace(x, 0);
    if (inserted) {
        it->second = compute(x);
    }
    return it->second;
}
```

Key Takeaway: Use `.find()` or `.try_emplace()` instead of `.count()` + `[]` to avoid redundant hash lookups.

topic_2_functor_pattern

Code Examples

📌 1. Basic Stateful Functor (Easy) (1/2)

Key Concepts:
- `operator()` overloading
- Internal state (`factor`, `callCount`)
- `mutable` keyword for modifying state in const methods
- STL algorithm compatibility

```
#include <iostream>
#include <vector>
#include <algorithm>

// Simple functor that multiplies values and tracks calls
class Multiplier {
    int factor;
    mutable int callCount;  // mutable to modify in const operator()

public:
    explicit Multiplier(int f) : factor(f), callCount(0) {}

    int operator()(int x) const {
        callCount++;
        return x * factor;
    }

    int getCallCount() const { return callCount; }
};

int main() {
    std::vector<int> numbers = {1, 2, 3, 4, 5};

    Multiplier times2(2);

    // Manual calls
    for (int n : numbers) {
        std::cout << times2(n) << " ";
    }
    std::cout << "\nCalls: " << times2.getCallCount() << "\n";

    // With STL algorithm
// ...continues on next slide
```

topic_2_functor_pattern

Code Examples

📌 1. Basic Stateful Functor (Easy) (2/2)

[Code continues across slides]

```
// ...continued from previous slide
    std::vector<int> result(numbers.size());
    std::transform(numbers.begin(), numbers.end(), result.begin(), Multiplier(3));

    for (int n : result) {
        std::cout << n << " ";
    }

    return 0;
}

// Output:
// 2 4 6 8 10
// Calls: 5
// 3 6 9 12 15
```

topic_2_functor_pattern

Code Examples

📌 2. Memoizing Functor (Mid) (1/3)

Key Concepts:
- Memoization pattern
- `mutable` for cache in const method
- Single lookup optimization with `.find()`
- Recursive functor calls with `(*this)(n)`
- Cache statistics tracking

```
#include <iostream>
#include <unordered_map>
#include <vector>

// Functor with memoization for expensive computations
class FibonacciCalculator {
    mutable std::unordered_map<int, long long> cache;
    mutable int cacheHits;
    mutable int cacheMisses;

public:
    FibonacciCalculator() : cacheHits(0), cacheMisses(0) {
        cache[0] = 0;
        cache[1] = 1;
    }

    long long operator()(int n) const {
        // ✅ Single lookup using find()
        auto it = cache.find(n);
        if (it != cache.end()) {
            cacheHits++;
            return it->second;
        }

        cacheMisses++;

        // Recursive computation using memoization
        long long result = (*this)(n - 1) + (*this)(n - 2);
        cache[n] = result;
        return result;
// ...continues on next slide
```

📌 2. Memoizing Functor (Mid) (2/3)

```
// ...continued from previous slide
    }

    void printStats() const {
        std::cout << "Cache size: " << cache.size() << "\n";
        std::cout << "Hits: " << cacheHits << ", Misses: " << cacheMisses << "\n";
        std::cout << "Hit rate: " << (100.0 * cacheHits / (cacheHits + cacheMisses)) << "%\n";
    }

    void clearCache() {
        cache.clear();
        cache[0] = 0;
        cache[1] = 1;
        cacheHits = 0;
        cacheMisses = 0;
    }
};

int main() {
    FibonacciCalculator fib;

    std::cout << "Computing Fibonacci numbers:\n";
    for (int i = 0; i <= 10; i++) {
        std::cout << "fib(" << i << ") = " << fib(i) << "\n";
    }

    fib.printStats();

    std::cout << "\nComputing again (should hit cache):\n";
    std::cout << "fib(10) = " << fib(10) << "\n";

    fib.printStats();

    return 0;
}

// Output:
// Computing Fibonacci numbers:
// fib(0) = 0
// fib(1) = 1
// fib(2) = 1
// ...continues on next slide
```

topic_2_functor_pattern

Code Examples

📌 2. Memoizing Functor (Mid) (3/3)

[Code continues across slides]

```
// ...continued from previous slide
// ...
// fib(10) = 55
// Cache size: 11
// Hits: 45, Misses: 11
// Hit rate: 80.36%
//
// Computing again (should hit cache):
// fib(10) = 55
// Cache size: 11
// Hits: 46, Misses: 11
// Hit rate: 80.70%
```

topic_2_functor_pattern

Code Examples

📌 3. Thread-Safe Functor (Advanced) (1/3)

Key Concepts:
- `std::mutex` for protecting shared cache
- `std::atomic` for counters (lock-free performance)
- Minimal lock scope (only around cache access)
- `std::ref()` to pass functor by reference to threads
- Thread-safe statistics collection

```
#include <iostream>
#include <vector>
#include <thread>
#include <mutex>
#include <atomic>
#include <unordered_map>
#include <functional>

// Thread-safe functor with memoization
class ThreadSafeSquare {
    mutable std::unordered_map<int, int> cache;
    mutable std::mutex cacheMutex;
    std::atomic<int> totalCalls{0};
    std::atomic<int> cacheHits{0};

public:
    int operator()(int x) {
        totalCalls++;

        // ✅ Try to read from cache with minimal locking
        {
            std::lock_guard<std::mutex> lock(cacheMutex);
            auto it = cache.find(x);
            if (it != cache.end()) {
                cacheHits++;
                return it->second;
            }
        }

        // Expensive computation (simulated with sleep)
// ...continues on next slide
```

📌 3. Thread-Safe Functor (Advanced) (2/3)

```
// ...continued from previous slide
        int result = x * x;

        // ✅ Write to cache
        {
            std::lock_guard<std::mutex> lock(cacheMutex);
            cache[x] = result;
        }

        return result;
    }

    void printStats() const {
        std::lock_guard<std::mutex> lock(cacheMutex);
        std::cout << "Total calls: " << totalCalls.load() << "\n";
        std::cout << "Cache hits: " << cacheHits.load() << "\n";
        std::cout << "Cache size: " << cache.size() << "\n";
    }
};

// Worker function for threads
void worker(ThreadSafeSquare& functor, const std::vector<int>& values) {
    for (int val : values) {
        int result = functor(val);
        // Process result...
    }
}

int main() {
    ThreadSafeSquare squareFunctor;

    // Create test data with duplicates
    std::vector<int> data1 = {1, 2, 3, 4, 5, 1, 2, 3};
    std::vector<int> data2 = {3, 4, 5, 6, 7, 3, 4};
    std::vector<int> data3 = {1, 5, 9, 1, 5, 9};

    // Launch threads using std::ref to avoid copying functor
    std::thread t1(worker, std::ref(squareFunctor), std::cref(data1));
    std::thread t2(worker, std::ref(squareFunctor), std::cref(data2));
    std::thread t3(worker, std::ref(squareFunctor), std::cref(data3));

// ...continues on next slide
```

📌 3. Thread-Safe Functor (Advanced) (3/3)

[Code continues across slides]

```
// ...continued from previous slide
    t1.join();
    t2.join();
    t3.join();

    std::cout << "\nThread-safe functor statistics:\n";
    squareFunctor.printStats();

    return 0;
}

// Output (typical):
// Thread-safe functor statistics:
// Total calls: 22
// Cache hits: 13
// Cache size: 9
```

topic_2_functor_pattern

Code Examples

📌 4. Functor with Generic Programming (Advanced) (1/3)

Key Concepts:
- Template functors for generic types
- SFINAE with `std::enable_if` for conditional methods
- Value initialization for generic default values
- Type traits (`std::is_arithmetic`)

```
#include <iostream>
#include <vector>
#include <string>
#include <type_traits>

// Generic functor that works with any type supporting operator+
template <typename T>
class Accumulator {
    T sum;
    int count;

public:
    Accumulator() : sum{}, count(0) {}  // Value initialization

    void operator()(const T& value) {
        sum += value;
        count++;
    }

    T getSum() const { return sum; }
    int getCount() const { return count; }

    // Average - only enabled for arithmetic types
    template <typename U = T>
    typename std::enable_if<std::is_arithmetic<U>::value, double>::type
    getAverage() const {
        return count > 0 ? static_cast<double>(sum) / count : 0.0;
    }
};

int main() {
    // Integer accumulator
// ...continues on next slide
```

topic_2_functor_pattern

Code Examples

📌 4. Functor with Generic Programming (Advanced) (2/3)

```
// ...continued from previous slide
    std::vector<int> numbers = {1, 2, 3, 4, 5};
    Accumulator<int> intAcc;

    for (int n : numbers) {
        intAcc(n);
    }

    std::cout << "Int sum: " << intAcc.getSum() << "\n";
    std::cout << "Int average: " << intAcc.getAverage() << "\n";

    // String accumulator
    std::vector<std::string> words = {"Hello", " ", "World", "!"};
    Accumulator<std::string> strAcc;

    for (const auto& word : words) {
        strAcc(word);
    }

    std::cout << "String concatenation: " << strAcc.getSum() << "\n";
    std::cout << "Word count: " << strAcc.getCount() << "\n";

    // Double accumulator
    std::vector<double> doubles = {1.5, 2.5, 3.5};
    Accumulator<double> doubleAcc;

    for (double d : doubles) {
        doubleAcc(d);
    }

    std::cout << "Double sum: " << doubleAcc.getSum() << "\n";
    std::cout << "Double average: " << doubleAcc.getAverage() << "\n";

    return 0;
}

// Output:
// Int sum: 15
// Int average: 3
// String concatenation: Hello World!
// Word count: 4
// ...continues on next slide
```

📌 4. Functor with Generic Programming (Advanced) (3/3)

[Code continues across slides]

```
// ...continued from previous slide
// Double sum: 7.5
// Double average: 2.5
```

topic_2_functor_pattern

Code Examples

📌 6. Functor Composition (Advanced) (1/3)

Key Concepts:
- Functor composition pattern
- Template deduction
- `decltype` for return type deduction
- Functional programming style in C++
- Reusable transformation pipelines

```
#include <iostream>
#include <functional>
#include <vector>
#include <algorithm>

// Composable functor wrapper
template <typename F, typename G>
class ComposedFunctor {
    F f;
    G g;

public:
    ComposedFunctor(F f_, G g_) : f(f_), g(g_) {}

    template <typename T>
    auto operator()(T x) const -> decltype(f(g(x))) {
        return f(g(x));  // f ∘ g
    }
};

// Helper function for composition
template <typename F, typename G>
ComposedFunctor<F, G> compose(F f, G g) {
    return ComposedFunctor<F, G>(f, g);
}

// Example functors
struct Square {
    int operator()(int x) const { return x * x; }
};
// ...continues on next slide
```

📌 6. Functor Composition (Advanced) (2/3)

```
// ...continued from previous slide

struct AddTen {
    int operator()(int x) const { return x + 10; }
};

struct MultiplyByTwo {
    int operator()(int x) const { return x * 2; }
};

int main() {
    Square square;
    AddTen addTen;
    MultiplyByTwo multiplyByTwo;

    // Compose: square(addTen(x))
    auto squareAfterAdd = compose(square, addTen);

    std::cout << "Square after adding 10:\n";
    std::cout << "f(5) = " << squareAfterAdd(5) << "\n";  // (5+10)^2 = 225

    // Compose: multiplyByTwo(square(x))
    auto doubleAfterSquare = compose(multiplyByTwo, square);

    std::cout << "Double after squaring:\n";
    std::cout << "f(5) = " << doubleAfterSquare(5) << "\n";  // (5^2)*2 = 50

    // Chain multiple compositions
    auto complex = compose(compose(multiplyByTwo, square), addTen);
    std::cout << "Complex composition:\n";
    std::cout << "f(5) = " << complex(5) << "\n";  // ((5+10)^2)*2 = 450

    // Use with STL algorithms
    std::vector<int> numbers = {1, 2, 3, 4, 5};
    std::vector<int> results(numbers.size());

    std::transform(numbers.begin(), numbers.end(), results.begin(), squareAfterAdd);

    std::cout << "Transformed vector: ";
    for (int n : results) {
        std::cout << n << " ";
// ...continues on next slide
```

📌 6. Functor Composition (Advanced) (3/3)

[Code continues across slides]

```
// ...continued from previous slide
    }
    std::cout << "\n";

    return 0;
}

// Output:
// Square after adding 10:
// f(5) = 225
// Double after squaring:
// f(5) = 50
// Complex composition:
// f(5) = 450
// Transformed vector: 121 144 169 196 225
```

What You Learned

Key Takeaways

- ✓ What is a Functor?
- ✓ Why Functors Matter in Autonomous Driving
- ✓ Functor vs Function Pointer vs Lambda
- ✓ Thread-Safety with Shared State

→ *Swipe to learn* →